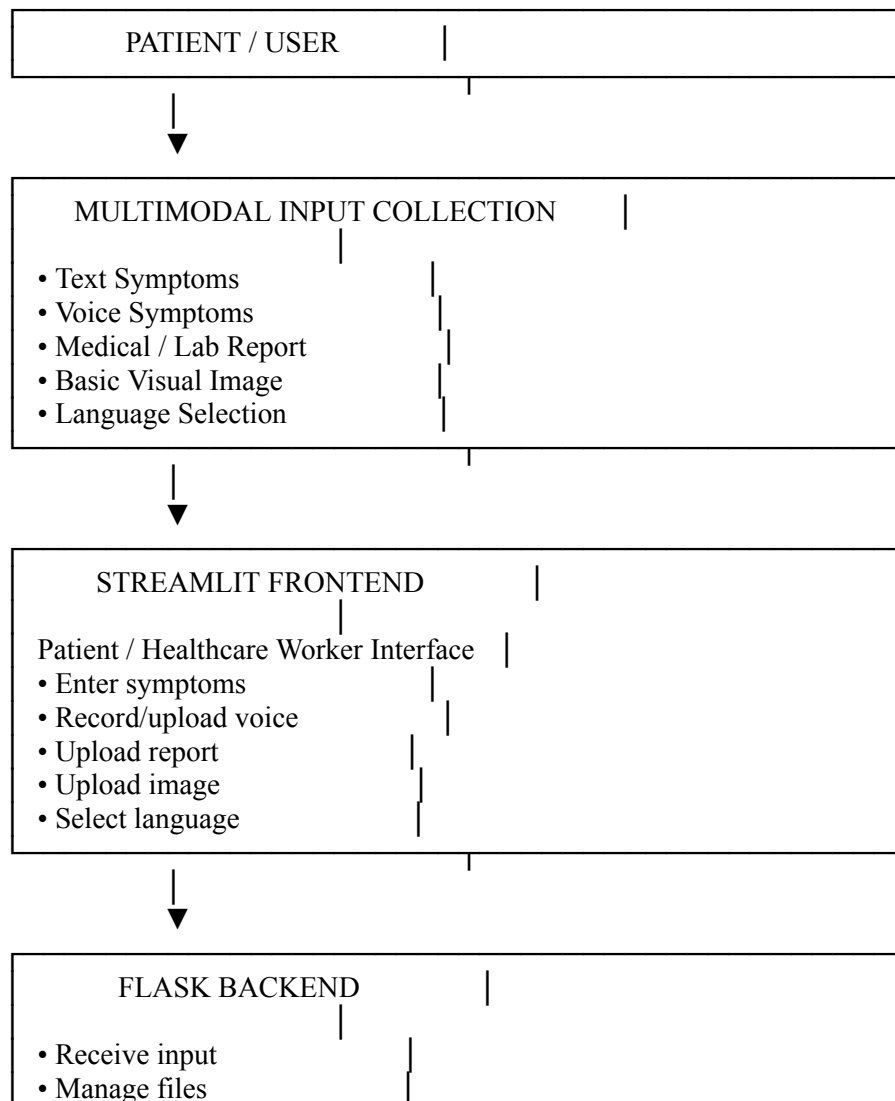
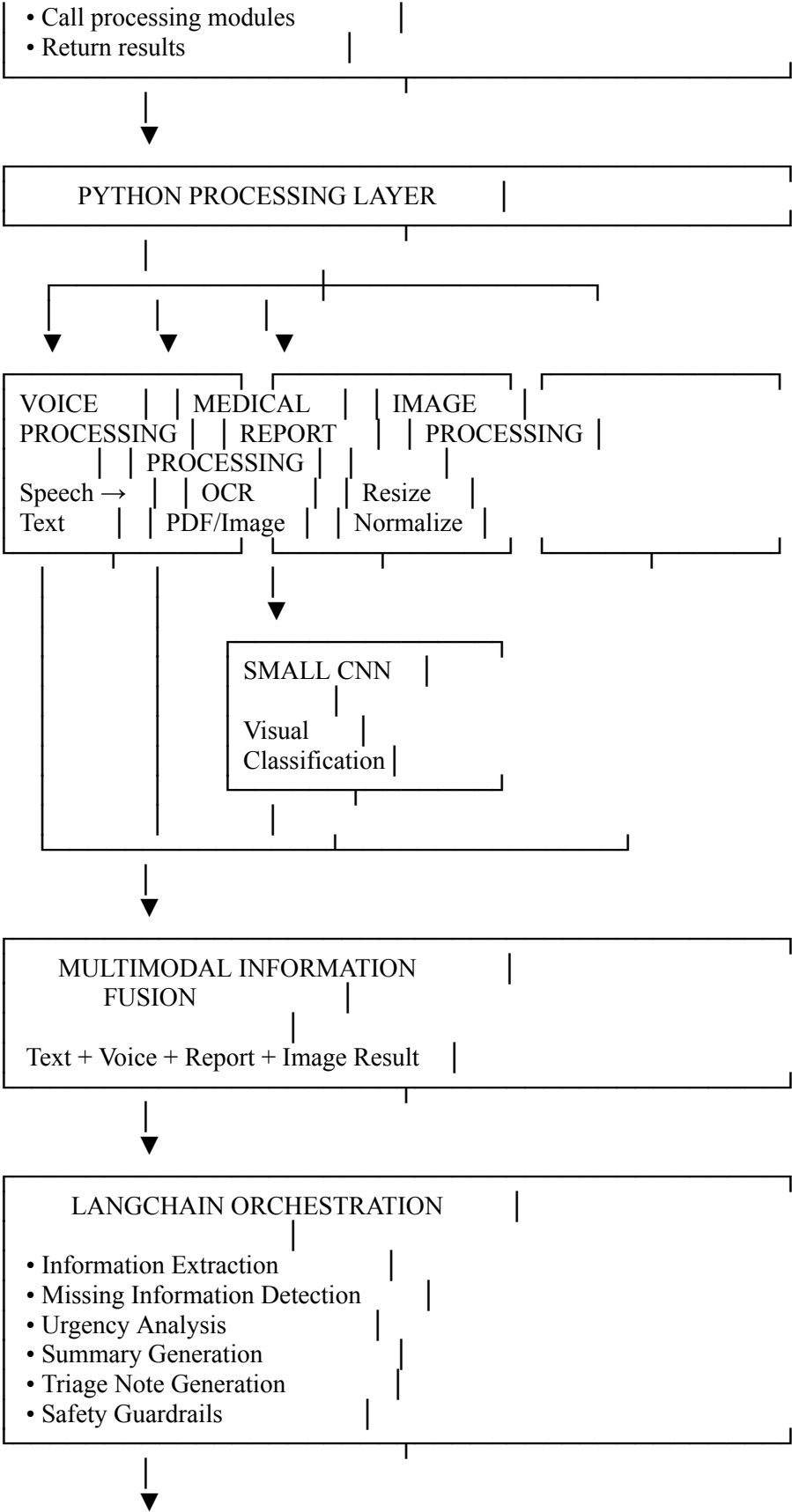


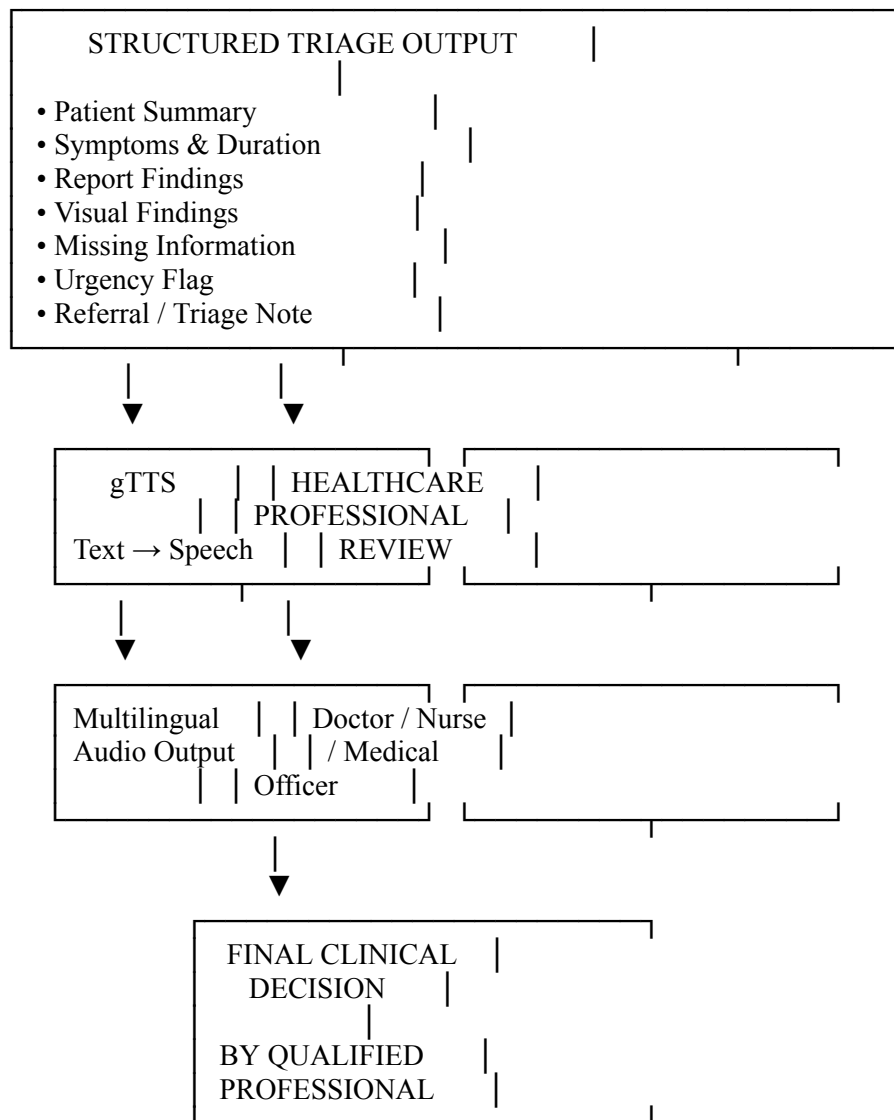
Multimodal Healthcare Triage Assistant — Architecture Flow

The architecture should be kept **simple, modular, and easy to explain during the demo**. The core idea is that the system receives **text, voice, medical reports, and basic images**, processes each input appropriately, combines the information, generates a **structured triage summary**, and sends it to a **qualified healthcare professional for the final decision**. This matches the project brief's intended workflow.

1. Overall Architecture Flow







2. Architecture in Simple Words

Layer 1 — Multimodal Input

The patient can provide information through:

- **Text**
- **Voice**
- **Medical/lab report**
- **Basic visual image**
- **English/Hindi/Odia/regional language**

The project specifically identifies these different forms of patient input.

Layer 2 — Streamlit Frontend

Streamlit is the user interface.

The interface provides:

Patient Input → Upload/Record → Analyze → Results

The healthcare worker can also use the interface to review the generated information.

Layer 3 — Flask Backend

Flask acts as the bridge between the frontend and AI system.

For example:

```
Streamlit
↓
Flask API
↓
Python AI modules
↓
Result
↓
Flask
↓
Streamlit
```

Suggested Flask APIs:

```
POST /ingest
POST /voice
POST /report
POST /image
POST /triage
GET /result
```

3. Python Processing Layer

Python processes each input according to its type.

Voice

Voice

↓
Speech-to-Text
↓
Text

Medical Report

PDF / Image
↓
OCR
↓
Extracted Text

Image

Image
↓
Resize
↓
Normalize
↓
Small CNN
↓
Visual Result

Language

English / Hindi / Odia / Regional
↓
Language Processing
↓
Normalized Information

4. Small CNN Model

The CNN should remain **small and lightweight** because this is a prototype.

Input Image
↓
Resize
↓
Normalization
↓
Conv Layer
↓
ReLU
↓
Pooling
↓

Conv Layer
↓
ReLU
↓
Pooling
↓
Dense Layer
↓
Visual Classification

The CNN output is then passed to the larger workflow.

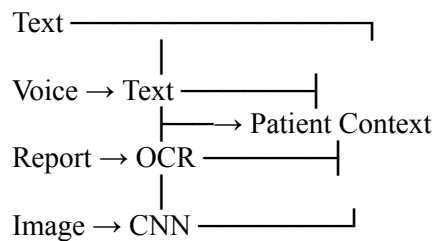
For example:

Image
↓
CNN
↓
Visual Finding / Risk Flag
↓
LangChain

It should **not independently produce a medical diagnosis**.

5. Multimodal Fusion

This is where all the information is brought together.



The system can create one structured object such as:

Patient Context

Symptoms:

Fever, weakness

Duration:

3 days

Report:

Extracted report information

Visual:
CNN visual result

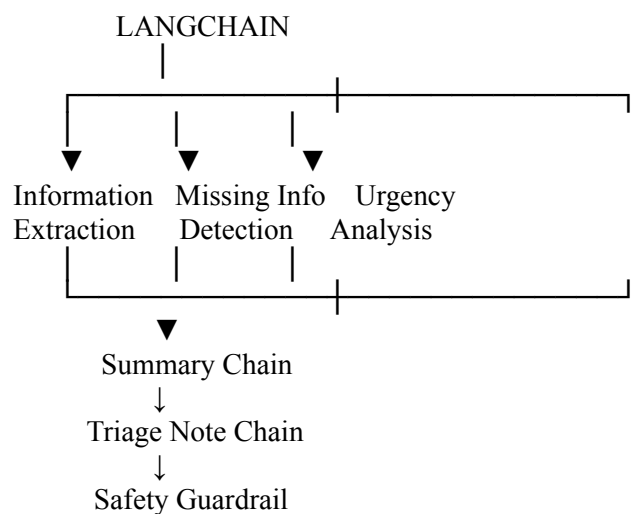
Language:
Hindi

Missing information:
Temperature, age, oxygen level

6. LangChain Orchestration

LangChain is the central workflow manager.

Instead of asking the LLM to do everything at once, divide the process into simple chains.



Information Extraction

Extract:

Complaint

Symptoms

Duration

Relevant report information

Missing Information

Identify information that may need to be collected by the healthcare worker.

Example:

Missing:

- Age
- Temperature
- Oxygen level

This matches the project's example of identifying missing information rather than inventing it.

Urgency Analysis

The system organizes the case into:

Routine Review

Needs Review

Priority Review

Urgent Professional Review

These categories are described in the project material.

Summary Generation

The AI generates a structured summary:

PATIENT SUMMARY

Complaint:
Fever

Duration:
3 days

Additional symptoms:
Weakness

Report information:
Extracted findings

Missing information:
Temperature
Age

Oxygen level

Urgency:

Needs prompt professional review

7. gTTS Output

After the structured result is generated:

Structured Text

↓

gTTS

↓

Audio

The audio can be played through Streamlit.

Example:

"Your information has been organized. Please wait for professional review."

So the purpose of **gTTS is accessibility**, especially for multilingual users.

8. Healthcare Professional Dashboard

The healthcare professional receives:

Patient Summary

Extracted Information

Missing Information

Urgency Flag

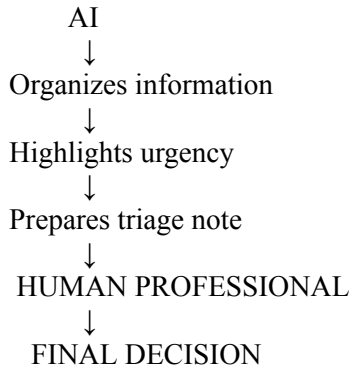
Referral/Triage Note

Audio Output

Then the professional reviews the information.

9. Final Decision

The final architecture must end with:



The project explicitly says that the AI should **not diagnose or prescribe treatment** and that the qualified healthcare professional makes the actual decision.

Final Architecture Statement

You can use this paragraph directly in your **project documentation/presentation**:

The proposed Multimodal Healthcare Triage Assistant follows a human-in-the-loop architecture that integrates multimodal patient inputs, including text symptoms, voice, medical reports, and basic visual information. A Streamlit-based frontend collects patient and healthcare-worker inputs, while a Flask backend manages communication between the interface and the Python-based processing modules. Voice inputs are converted into text, uploaded reports are processed using OCR, and visual inputs are analyzed using a lightweight CNN model. The processed information is then combined into a unified patient context and passed to a LangChain-based orchestration layer. LangChain coordinates information extraction, missing-information detection, urgency analysis, structured summarization, and triage-note generation. The resulting structured information is presented through the healthcare-worker dashboard and can also be converted into multilingual audio using gTTS. Finally, a qualified doctor, nurse, or medical officer reviews the AI-generated information and makes the final clinical decision. The system is therefore designed as a triage and information-assistance tool rather than a diagnostic or prescription system.

One-line flow for your architecture figure

Patient → Multimodal Input → Streamlit → Flask → Python Preprocessing → OCR / Speech-to-Text / CNN → Multimodal Fusion → LangChain → Summary + Missing Information + Urgency → Structured Triage → gTTS + Healthcare Dashboard → Doctor/Nurse Review → Final Decision